

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

В данной статье представлено формальное описание операционной семантики выражений языка программирования Go с использованием языка спецификаций ABML. Рассматривается фрагмент языка Go, не включающий в себя работу с многопоточностью и соответствующими конструкциями (горутины, каналы, операторы `go` и `select`). Предложенная онтологическая модель позволяет строго задать семантику базовых конструкций языка и может служить основой для разработки инструментов верификации программ.

Ключевые слова: операционная семантика, язык Go, язык ABML, формальные методы, онтологическое моделирование.

1. Введение

Формальное описание семантики языков программирования является фундаментальной задачей, лежащей в основе разработки компиляторов, статических анализаторов и инструментов верификации программ. Традиционные подходы к спецификации семантики, основанные на естественном языке, часто страдают от неоднозначности и неполноты, что затрудняет создание формально верифицируемых инструментов.

В данной работе рассматривается фрагмент языка программирования Go и его операционная семантика, формализованная с помощью языка спецификаций ABML (Attribute-Based Modeling Language), который является лексическим расширением диалекта Common Lisp — SBCL. Выбор языка Go обусловлен его возрастающей популярностью в промышленной разработке, особенно в области системного программирования. Язык ABML, в свою очередь, предоставляет средства для описания онтологий предметных областей и формализации правил перехода между состояниями.

2. Онтология фрагмента языка Go

Построение формальной модели семантики языка программирования требует предварительного создания онтологии — системы понятий, описывающих синтаксические конструкции и семантические сущности языка. В данном разделе на языке ABML вводятся типы объектов, соответствующие элементам фрагмента языка Go.

Классификация и порядок изложения опираются на официальную спецификацию языка Go. Большинство наименований моделируемых объектов соответствуют их названиям в спецификации. Однако, часть конструкций языка или их составляющих не имеет уникальных наименований, или их названия связаны с синтаксисом, что не позволяет достаточно точно определять их смысл в различных контекстах.

2.1. Имена

Все имена в программе на Go моделируются как строки. Вводятся следующие типы имен:

```
(typedef "identifier"      (uniont string))
(typedef "constant name"  "identifier")
(typedef "variable name"  "identifier")
(typedef "field name"     "identifier")
(typedef "function name"  "identifier")
(typedef "parameter name" "identifier")
(typedef "method name"    "identifier")
(typedef "label name"     "identifier")
(typedef "type name"      "identifier")
```

2.2. Типы данных

Система типов языка Go разделяется на базовые (простые) и составные типы.

```
(typedef "type" (uniont "primitive type" "composite type"))
```

2.2.1. Базовые типы

Базовые типы включают логический, строковый и числовые.

```
(typedef "primitive type" (uniont "bool type" "string type" "numeric
    type"))
(cot "bool type")
(cot "string type")
```

Числовые типы детализируются. Целые типы разделяются на знаковые и беззнаковые с указанием разрядности. Вещественные и комплексные типы также имеют разрядность.

```
(typedef "numeric type" (uniont "int type" "float type" "complex
    type"))
(typedef "int type" (uniont "signed int type" "unsigned int type"))
(cot "signed int type" :at "bit size" (enumt 8 16 32 64))
(cot "unsigned int type" :at "bit size" (enumt 8 16 32 64))
(cot "float type" :av "bit size" (enumt 32 64))
(cot "complex type" :av "bit size" (enumt 64 128))
```

2.2.2. Составные типы

Тип составных объектов включает в себя типы массивов, срезов, структур, указателей, функций, методов, интерфейсов, карт и каналов.

```
(typedef "composite type" (uniont "array type" "slice type" "struct
    type"
    "pointer type" "function type"
    "method type"
    "interface type" "map type" "channel
    type"))
```

Тип массивов содержит информацию о типе элементов и фиксированной длине. Срезы — только тип элементов, поскольку длина среза определяется динамически.

```
(cot "array type" :at "elem type" "type" :at "len" nat)
(cot "slice type" :at "elem type" "type")
```

Структура определяется набором полей. Атрибут `ordered` используется на этапе статического анализа для поддержки декларации экземпляра структуры с позиционным заданием значений полей.

```
(cot "struct type" :at "fields" (cot :amap "field name" "type")
    :at "ordered" (listt "field name"))
```

Тип указателей хранит информацию о типе, на который он указывает.

```
(cot "pointer type" :at "type" "type")
```

Функциональный тип описывает сигнатуру функции. Имена параметров опущены, так как при проверке типов важны только соответствующие типы. Вариативный параметр, позволяющий функции принимать произвольное число аргументов (задается многоточием), моделируется отдельным атрибутом.

```
(cot "function type"
    :at "param types" (listt "type")
    :at "variadic type" "type"
    :at "result types" (listt "type"))
```

Тип методов содержит получателя (receiver) и используется, в том числе, при определении интерфейсов.

```
(cot "method type"
    :at "receiver type" "type"
    :at "param types" (listt "type")
    :at "variadic type" "type"
    :at "result types" (listt "type"))
```

Тип интерфейсов определяется как набор реализуемых методов с их сигнатурами.

```
(cot "interface type" :at "methods" (cot :amap "method name" "method
```

```
type"))
```

Тип карт задается типами ключа и значения.

```
(cot "map type" :at "key type" "type" :at "elem type" "type")
```

Канал имеет тип передаваемых элементов. Направление не моделируется, так как для корректных программ оно не влияет на семантику.

```
(cot "channel type" :at "elem type" "type")
```

2.3. Ячейки памяти и значения

В данной работе модель памяти представлена типизированными значениями. То есть каждое значение имеет собственный тип, известный еще до этапа выполнения программы.

Ячейка представляет собой изменяемый контейнер, хранящий значение. Переменные связываются с ячейками, а не непосредственно со значениями. Тип ячейки указывает на тип хранимого значения.

```
(mot "cell" :at "value" "Go value" :at "type" "type")
(typedef "Go value" (uniont "untyped constant" "typed primitive"
  "composite value"))
```

Нетипизированные константы существуют только на этапе статического анализа. До начала выполнения программы в операционной семантике они преобразуются в типизированные значения.

```
(typedef "untyped constant" (uniont bool int real string complex))
```

Типизированные примитивы моделируют простейшие значения, то есть для хранения которых не используются ячейки памяти, с их статическими типами. Такими значениями являются строки и числа, в том числе вещественные и комплексные.

```
(mot "typed primitive"
```

```

:at "type" "type"
:at "value" (uniont "nil" "untyped constant"))

```

Nil-значение, что встречается в предыдущей конструкции, является значением сложных типов при инициализации. Стоит заметить, что nil-ы разных типов различаются.

```

(cot "nil")

```

Составные значения моделируют структуры данных языка Go.

```

(typedef "composite value" (uniont "array value" "slice value" "struct
value"
                                "map value" "interface value"
                                "method value"
                                "function value" "cell" "channel
value"))

```

Массив хранит элементы в виде списка ячеек, что обеспечивает возможность изменения элементов. Однако, длина массива определяется собственным типом, и изменяться не может.

```

(mot "array value"
  :at "type" "array type"
  :at "elements" (listt "cell"))

```

Срез является представлением фрагмента массива. Несколько срезов могут разделять один и тот же массив.

```

(mot "slice value"
  :at "type" "slice type"
  :at "array" "array value"
  :at "offset" nat
  :at "length" nat
  :at "capacity" nat)

```

Структура хранит поля как отображение имен на ячейки.

```
(mot "struct value"
  :at "type"    "struct type"
  :at "fields" (mot :amap "field name" "cell"))
```

Карта хранит записи как отображение ключей на ячейки значений.

```
(mot "map value"
  :at "type"    "map type"
  :at "entries" (mot :amap "Go value" "cell"))
```

Канал представляет собой FIFO-очередь с буфером и очередями ожидающих горутин (моделируемых ячейками).

```
(mot "channel value"
  :at "type"          "channel type"
  :at "buffer"        (listt "Go value")
  :at "send queue"    (listt "cell")
  :at "receive queue" (listt "cell")
  :at "closed"        bool)
```

Функция хранит тип, сигнатуру (с именами параметров), тело и замыкание — отображение захваченных переменных на ячейки.

```
(mot "function value"
  :at "type"          "function type"
  :at "signature"     "function signature"
  :at "body"          "block"
  :at "closure"       (mot :amap "variable name" "cell"))
```

Сигнатура функции содержит список деклараций параметров, вариативный параметр (если есть) и результаты (могут быть безымянными — только типы, или именованными).

```
(mot "function signature"
```

```

:at "parameters"      (listt "param decl")
:at "variadic param"  "param decl"
:at "results"         (uniont (listt "type") (listt "param decl")))

(mot "param decl" :at "type" "type" :at "name" "parameter name")

```

Метод аналогичен функции, но имеет отдельный получатель. Сигнатура метода включает получатель как отдельную декларацию параметра.

```

(mot "method value"
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block"
  :at "closure"   (mot :amap "variable name" "cell"))

(mot "method signature"
  :at "receiver"      "param decl"
  :at "parameters"    (listt "param decl")
  :at "variadic param" "param decl"
  :at "results"       (listt "param decl"))

```

Интерфейс хранит статический тип и значение. Динамический тип содержится внутри значения.

```

(mot "interface value"
  :at "type"  "interface type"
  :at "value" "typed value")

```

2.4. Литералы

Литералы представляют собой значения, которые могут быть непосредственно записаны в исходном коде. Общий тип литерала объединяет типизированные константы и составные литералы.

```
(typedef "literal" (uniont "untyped constant" "numeric lit"
                        "composite lit" "function lit" "method lit"))
```

Числовые литералы используются для задания чисел с помощью различных представлений.

```
(mot "numeric lit" :at "value" string)
```

Литералы для составных значений: массив, срез, структура или карта.

```
(typedef "composite lit" (uniont "array lit" "slice lit"
                                "struct lit" "map lit"))
```

Литералы массивов и срезов могут задаваться с явным указанием индекса для каждого значения. Каждый такой элемент представлен парой `index & elem`.

```
(mot "index & elem" :at "index" nat :at "value" "expression")
```

```
(mot "array lit"
    :at "type"      "array type"
    :at "elements" (listt "index & elem"))
```

```
(mot "slice lit"
    :at "type"      "slice type"
    :at "elements" (listt "index & elem"))
```

Литералы структур могут объявляться с явным указанием имени поля для каждого значения. Каждый такой элемент представлен парой `field & value`.

```
(mot "field & value" :at "field" "field name" :at "value" "expression")
```

```
(mot "struct lit"
    :at "type"      "struct type"
    :at "fields" (listt "field & value"))
```

Литералы карт объявляются явным перечислением пар ключ-значение.

```
(mot "key & elem" :at "key" "expression" :at "value" "expression")

(mot "map lit"
  :at "type"      "map type"
  :at "elements" (listt "key & elem"))
```

Функциональные литералы (анонимные функции) хранят тип, сигнатуру и тело.

```
(mot "function lit"
  :at "type"      "function type"
  :at "signature" "function signature"
  :at "body"      "block")

(mot "method lit"
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block")
```

2.5. Выражения

Выражения в языке Go — это конструкции, которые после вычисления возвращают значение. Все выражения содержат атрибут `type`, значение которого вычисляется на этапе статического анализа.

```
(typedef "expression" (uniont "literal" "variable ref" "(1)"
  "conversion" "method expr" "selector expr" "index expr"
  "slice expr" "type assertion" "<-1" "function call"
  "unary expression" "binary expression"))
```

Ссылка на переменную представляет использование уже объявленной переменной.

```
(mot "variable ref" :at "name" "variable name" :at "type" "type")
```

Выражение в скобках влияет только на порядок вычисления.

```
(mot "(1)" :at 1 "expression" :at "type" "type")
```

Приведение типов позволяет явно преобразовывать значения.

```
(mot "conversion" :at "type" "type" :at "value" "expression")
```

Метод-выражение возвращает функцию с получателем в качестве первого параметра.

```
(mot "method expr" :at "receiver type" "type"
      :at "name" "method name"
      :at "type" "type")
```

Выбор поля или метода по экземпляру.

```
(mot "selector expr" :at "receiver" "expression"
      :at "name" "identifier"
      :at "type" "type")
```

Индексирование применимо к массивам, срезам, картам и строкам.

```
(mot "index expr" :at "indexable" "expression"
      :at "index" "expression"
      :at "type" "type")
```

Выражение среза создает новый срез из существующего массива, строки или среза. Параметры `low`, `high` и `max` задают границы. Последний из них может отсутствовать.

```
(mot "slice expr"
      :at "sequence" "expression"
      :at "low"      "expression"
      :at "high"     "expression")
```

```

:at "max"      "expression"
:at "type"     "slice type")

```

Проверка типа применяется к интерфейсным значениям.

```

(mot "type assertion" :at "interface" "expression" :at "type" "type")

```

Операция чтения из канала.

```

(mot "<-1" :at 1 "expression" :at "type" "type")

```

Вызов функции и метода.

```

(mot "function call" :at "function" "expression"
      :at "arguments" (listt "expression")
      :at "type" "type")

(mot "method call" :at "receiver" "expression"
      :at "method" "method name"
      :at "arguments" (listt "expression")
      :at "type" "type")

```

Унарные выражения используют цифровую нотацию для обозначения операнда.

```

(typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1" "&1"))

(mot "+1" :at 1 "expression" :at "type" "type")
(mot "-1" :at 1 "expression" :at "type" "type")
(mot "^1" :at 1 "expression" :at "type" "type")
(mot "!1" :at 1 "expression" :at "type" "type")
(mot "*1" :at 1 "expression" :at "type" "type")
(mot "&1" :at 1 "expression" :at "type" "type")

```

Бинарные выражения также используют цифровую нотацию: **1** — левый операнд, **2** —

правый.

```
(typedef "binary expression" (uniont "1||2" "1&&2" "1+2" "1-2" "1*2"
                                     "1/2" "1%2" "1<<2" "1>>2" "1&2"
                                     "1^2" "1|2" "1^2" "1==2" "1!=2"
                                     "1<2" "1<=2" "1>2" "1>=2"))
```

2.6. Блоки

Блоки определяют области видимости переменных и меток. В языке Go блок представляет собой последовательность операторов, заключенную в фигурные скобки.

```
(mot "block"
  :at "statements" (listt "statement")
  ;; semantic attributes
  :at "label positions" (mot :amap "label name" nat)
  :at "all variables" (listt "variable name")
  :at "decl variables" (listt "variable name")
  :at "variable cells" (mot :amap "variable name" "cell"))
```

Атрибут `statements` содержит список операторов блока. Атрибут `label positions` предоставляет меткам их позиции в блоке. Атрибут `variable cells` сохраняет для каждой переменной ее ячейку памяти до входа в блок, что позволяет корректно обрабатывать затенение имен переменных.

2.7. Декларации

Декларации вводят новые объекты в программу.

```
(typedef "declaration" (uniont "type decl" "const decl" "var decl"
                              "function decl" "method decl"))
```

Декларация типа связывает имя с определением типа.

```
(mot "type decl" :at "name" "type name" :at "type" "type")
```

Декларация констант и переменных может содержать явное указание типов (опционально) и значения.

```
(mot "const decl"
  :at "names" (listt "constant name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))

(mot "var decl"
  :at "names" (listt "variable name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))
```

Декларация функций и методов.

```
(mot "function decl" :at "name" "function name" :at "value" "function
  lit")
(mot "method decl" :at "name" "method name" :at "value" "method
  lit")
```

2.8. Операторы

Операторы в языке Go представляют собой конструкции, которые изменяют состояние программы, но не возвращают значений.

```
(typedef "statement" (uniont "declaration" "label" "empty stmt" "1++"
  "1—" "assignment stmt" "if stmt"
  "switch stmt" "for stmt" "return stmt"
  "break stmt" "continue stmt"
  "goto stmt" "defer stmt" "go stmt"
  "1<-2" "fallthrough" "select stmt"))
```

Помеченный оператор позволяет присвоить метку любому оператору программы.

```
(mot "label" :at "name" "label name" :at "statement" "statement")
```

Пустой оператор не выполняет никаких действий.

```
(cot "empty stmt")
```

Операторы инкремента и декремента являются операторами, а не выражениями, что отличает Go от многих других языков.

```
(mot "1++ stmt" :at 1 "expression")  
(mot "1-- stmt" :at 1 "expression")
```

Операторы присваивания включают как простое присваивание, так и составные с арифметическими и битовыми операциями.

```
(typedef "assignment stmt" (uniont "1=2" "1+=2" "1-=2" "1*=2" "1/=2"  
                                     "1%=2" "1<<=2" "1>>=2" "1^=2"  
                                     "1|=2" "1&=2" "1&^=2"))  
  
(mot "1=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1+=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1-=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1*=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1/=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1%=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1<<=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1>>=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1^=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1|=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1&=2" :at 1 (listt "expression") :at 2 (listt "expression"))  
(mot "1&^=2" :at 1 (listt "expression") :at 2 (listt "expression"))
```

Условный оператор может содержать необязательную инициализирующую часть.

```
(mot "if stmt"
  :at "init"      "statement"
  :at "condition" "expression"
  :at "then"      "block"
  :at "else"      (uniont "if stmt" "block"))
```

Оператор выбора существует в двух формах.

```
(typedef "switch stmt" (uniont "expr switch stmt" "type switch stmt"))

(cot "default")
(cot "fallthrough")
```

Первая, переключатель по выражению.

```
(mot "expr switch stmt"
  :at "init"      "statement"
  :at "controlled" "expression"
  :at "cases"      (listt "expr case clause"))

(mot "expr case clause"
  :at "cases"      (uniont (listt "expression") "default")
  :at "statements" (listt "statement"))
```

Вторая, переключатель по типу.

```
(mot "type switch stmt"
  :at "init"      "statement"
  :at "name"      "variable name"
  :at "value"     "expression"
  :at "cases"     (listt "type case clause"))

(mot "type case clause"
```

```

:at "cases"      (uniont (listt "type") "default")
:at "statements" (listt "statement"))

```

Оператор цикла может быть представлен в нескольких формах.

```

(typedef "for stmt" (uniont "for condition" "for clause"
                           "for range indexable" "for range map"
                           "for range channel" "for range int"))

```

Классический цикл с условием. В языке программирования C и ему подобных аналогичная конструкция обозначается ключевым словом `while`.

```

(mot "for condition"
  :at "condition" "expression"
  :at "body"      "block")

```

Цикл с инициализацией, условием и пост-действием.

```

(mot "for clause"
  :at "init"      "statement"
  :at "condition" "expression"
  :at "post"      "statement"
  :at "body"      "block")

```

Цикл `range` для массивов, срезов или строк.

```

(mot "for range indexable"
  :at "index"      "variable name"
  :at "value"      "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "indexable" "expression"
  :at "body"      "block")

```

Цикл `range` для карт.

```
(mot "for range map"
  :at "key"      "variable name"
  :at "value"    "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "map"      "expression"
  :at "body"     "block")
```

Цикл **range** для каналов.

```
(mot "for range channel"
  :at "value"    "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "channel"  "expression"
  :at "body"     "block")
```

Цикл **range** для перечисления отрезка натурального ряда до заданного числа.

```
(mot "for range int"
  :at "value"    "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "max"      "expression"
  :at "body"     "block")
```

Операторы передачи управления включают **return**, **break**, **continue** и **goto**.

```
(mot "return stmt"  :at 1 (listt "expression"))
(mot "break stmt"   :at 1 "label name")
(mot "continue stmt" :at 1 "label name")
(mot "goto stmt"    :at 1 "label name")
```

Оператор **defer** откладывает выполнение функции до возврата из окружающей функции. Оператор **go** запускает функцию в отдельной горутине.

```
(mot "defer stmt" :at 1 "expression")
```

```
(mot "go stmt" :at 1 "expression")
```

Оператор отправки данных в канал.

```
(mot "l<-r" :at 1 "expression" :at 2 "expression")
```

Оператор `select` для работы с несколькими каналами.

```
(mot "select stmt" :at "cases" (listt "common clause"))

(mot "common clause"
  :at "case" (uniont "send stmt" "receive assign" "receive
decl" "default"))
  :at "statements" (listt "statement"))

(mot "receive assign" :at 1 (listt "expression") :at 2 "expression")
(mot "receive decl" :at 1 (listt "variable name") :at 2 "expression")
```

3. Операционная семантика выражений

Операционная семантика определяет, как вычисляются выражения языка Go в процессе выполнения программы. В данном разделе на языке ABML формализуются правила вычисления для всех конструкций языка Go.

Вычисление происходит в контексте окружения — агента, который хранит информацию о переменных, константах, функциях и типах. Каждый шаг вычисления представлен декларацией аспекта (`aspect`), которая задает атрибут `opsem::rvalue` для выражений, возвращающих значения, `opsem::lvalue` для получения ячеек памяти, и `opsem` для конструкций языка, не возвращающих значений.

3.1. Окружение и агенты

Окружение представляет собой совокупность агентов, каждый из которых моделирует состояние выполнения программы. Атрибут `value` автоматически заполняется послед-

ним вычисленным этим агентом значением. Остальные 5 атрибутов хранят отношения декларируемых в программе имен к значениям констант, переменных, функций, методов и пользовательских типов соответственно.

```
(mot "env" :at "agents" (listt "agent"))

(mot "agent"
  :at "constant value" (mot :amap "constant name" "Go value")
  :at "variable cell" (mot :amap "variable name" "cell")
  :at "function value" (mot :amap "function name" "function value")
  :at "method value" (mot :amap "type name"
                          (mot :amap "method name" "method value"))
  :at "type" (mot :amap "type name" "type")
  :at "value" "Go value")
```

3.2. Базовое правило для значений

Если выражение уже является значением, оно возвращается без изменений.

```
(aspect "opsem::rvalue" :context ac :type "Go value" :instance i :do i)
```

3.3. Литералы

Здесь и далее будет описываться вычисление соответствующих конструкций онтологической модели. Повторно приводиться их описание не будет.

3.3.1. Литерал массива

Вычисление литерала массива происходит в несколько этапов. Сначала вычисляется значение по умолчанию для элементов массива. Соответствующая информация получается из типа литерала.

```
(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
  :stage nil :do
```

```
(update-push-acontext ac :stage "fill defaults")
(clear-update-eval-acontext ac :aspect "default value by type"
 :instance (aget i "type" "elem type"))
```

Затем, на стадии `fill defaults` создается массив из ячеек памяти, содержащих значения по умолчанию.

```
(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
 :stage "fill defaults" :ap i "elements" es :ap i "type" at
 :ap at "elem type" et :p (mo "pointer type" :av "type" et) ct
 :value v :do

(match :v (null es) T
 :do (mo "array value" :av "type" at :av "elements" nil)
 :exit (update-push-acontext ac :stage "evaluating" :av "left" es
 :av "array" (let ((c nil))
 (dotimes (k (aget at "len") c) (setf c
 (cons (mo "cell" :av "value" v :av "type" ct) c))))))
 (clear-update-eval-acontext ac
 :instance (aget (car es) "value"))))
```

На стадии `evaluating` происходит замена значений в соответствующих ячейках массива. Сначала изменяет k-й элемент массива. Затем, если вычислять нечего, то создает и возвращает значение массива. Иначе рекурсивно вызывается этот же шаг для необработанных элементов.

```
(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
 :stage "evaluating" :ap ac "left" left :ap ac "array" arr
 :value v :ap (car left) "index" k :do

(let ((c arr)) (dotimes (j k (setf (car c) "value" v))
 (setf c (cdr c))))
```

```
(match :v (null (cdr left)) T
:do (mo "array value" :av "type" (aget i "type") :av "elements" arr)
:exit (update-push-acontext ac :av "left" (cdr left)
      :av "array" arr)
      (clear-update-eval-acontext ac
      :instance (aget (car (cdr left)) "value"))))
```

3.3.2. Литерал среза

Вычисление литерала среза происходит аналогично вычислению массива. Сначала вычисляется значение по умолчанию.

```
(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
:stage nil :do

(update-push-acontext ac :stage "build value")
(update-push-acontext ac :stage "fill defaults")
(clear-update-eval-acontext ac :aspect "default value by type"
:instance (aget i "type" "elem type")))
```

На стадии `fill defaults` по максимальному индексу `m` создается нужного размера массив, заполненный ячейками со значениями по умолчанию.

```
(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
:stage "fill defaults" :value v :ap i "elements" es :p -1 m
:p (mo "pointer type" :av "type" (mo "pointer type"
:av "type" (aget i "type" "elem type"))) ct :do

(dolist (e es) (let ((k (aget e "index"))) (if (> k m) (setf m k))))
(match :v (null es) T
:do nil
:exit (update-push-acontext ac :stage "evaluating" :av "left" es
      :av "array" (let ((a nil)) (dotimes (k (1+ m)) a))
```

```

      (setf a (cons (mo "cell" :av "value" v :av "type" ct) a))))
    (clear—update—eval—acontext ac
      :instance (aget (car es) "value"))))

```

На стадии `evaluating`, после вычисления всех явно заданных элементов, возвращает их список в правильном порядке.

```

(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
  :stage "evaluating" :ap ac "left" left :ap ac "array" arr
  :value v :p (aget (car left) "index") k :do

  (let ((c arr))
    (dotimes (j k (setf (car c) "value" v)) (setf c (cdr c))))
  (match :v (null left) T
    :do (reverse arr)
    :exit (update—push—acontext ac :av "left" (cdr left) :av "array" arr)
      (clear—update—eval—acontext ac
        :instance (aget (car (cdr left)) "value"))))

```

На последней стадии `build value` создается значение среза, ссылающееся на созданный массив.

```

(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
  :stage "build value" :value arr :ap i "type" stp
  :ap stp "elem type" etp :p (length arr) n :do

  (mo "slice value"
    :av "type" (aget i "type")
    :av "array" (mo "array value" :av "type" (mo "array type"
      :av "elem type" etp :av "len" n) :av "elements" arr)
    :av "offset" 0
    :av "length" n
    :av "capacity" n))

```

3.3.3. Литерал структуры

Если структура имеет хотя бы одно поле, то вычисляет значение по умолчанию для первого поля структуры и отправляет на следующую стадию.

```
(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage nil :p (aget i "type" "fields") tfs
  :p (attributes tfs) tns :do

  (update-push-acontext ac :stage "build value")
  (update-push-acontext ac :stage "eval field values")
  (match :v (null ns) T :do nil
  :exit (update-push-acontext ac
    :stage "filling defaults"
    :av "type field names" tns
    :av "struct value" (mo :amap "field name" "cell"))
  (clear-update-eval-acontext ac
    :aspect "default value by type"
    :instance (aget tfs (car tns))))))
```

На стадии `filling defaults` все поля получают значения по умолчанию в соответствии с собственным типом.

```
(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "filling defaults" :ap ac "type field names" tns
  :ap ac "struct value" sv :value dv
  :p (mo "pointer type" :av "type" (mo "pointer type" :av "type"
    (aget i "type" "fields" (car tns)))) ct :do

  (aset sv (car tns) (mo "cell" :av "value" dv :av "type" ct))
  (match :v (null tns) T
  :do sv
  :exit (update-push-acontext ac :av "type field names" (cdr tns)))
```

```

      :av "struct value" sv)
  (clear—update—eval—acontext ac :aspect "default value by type"
    :instance (aget (aget i "type" "fields") (car (cdr tns))))))

```

На стадии `eval field values` все явно перечисленные поля получают заданные значения.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "eval field values" :value sv
  :ap i "fields" fs :p (attributes fs) ns :do

  (match :v (null ns) T
    :do sv
    :exit (update—push—acontext ac :stage "evaluating" :av "names" ns
      :av "struct value" sv)
    (clear—update—eval—acontext ac :instance (aget fs (car ns))))))

```

На стадии `evaluating` вычисляется очередное явно заданное значение поля структуры.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "evaluating" :ap ac "names" ns :ap ac "struct value" sv
  :ap i "fields" fs :value v :do

  (aset sv (car ns) "value" v)
  (match :v (null (cdr ns)) T
    :do sv
    :exit (update—push—acontext ac :av "names" (cdr ns)
      :av "struct value" sv)
    (clear—update—eval—acontext ac
      :instance (aget fs (car (cdr ns))))))

```

На стадии `build value` создается и возвращается значение структуры.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i

```

```

:stage "build value" :value sv :do

(mo "struct value" :av "type" (aget i "type") :av "fields" sv))

```

3.3.4. Литерал карты

Вычисление литерала карты начинается с последовательного вычисления пар ключ-значение.

```

(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage nil :ap i "elements" es :do

  (update-push-acontext ac :stage "build value")
  (match :v (null es) T
    :do nil
    :exit (update-push-acontext ac
      :stage "evaluating"
      :av "entries" (mo :amap "Go value" "cell")
      :ap "elements" (cdr es))
      (clear-update-eval-acontext ac :instance (car es)
        :av "elem type" (aget i "type" "elem type")))))

```

На стадии `evaluating` добавляются вычисленные записи.

```

(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage "evaluating" :ap ac "entries" ent
  :ap ac "elements" es :value ke :do

  (aset ent (aget ke "key") (aget ke "value"))
  (match :v (null es) T :do ent
    :exit (update-push-acontext ac :av "entries" ent
      :av "elements" (cdr es))
      (update-push-acontext ac :instance (car es))
      (clear-update-eval-acontext ac :instance (aget (car es) "key")

```

```
:av "elem type" (aget i "type" "elem type")))
```

На стадии `build value` создается и возвращается значение карты.

```
(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage "build value" :value ent :do

  (mo "map value" :av "type" (aget i "type") :av "entries" ent))
```

Теперь разберем вычисление записей карты. Оно начинается с вычисления значения.

```
(aspect "opsem::rvalue" :context ac :type "keyed elem" :instance i
  :stage nil :value k :do

  (update-push-acontext ac :stage "eval value" :av "key" k)
  (clear-update-eval-acontext ac :instance (aget i "value")))
```

Затем, на стадии `eval value` создается и возвращается запись карты.

```
(aspect "opsem::rvalue" :context ac :type "keyed elem" :instance i
  :stage "eval value" :ap ac "key" k :ap ac "elem type" et :value v :do

  (mo "keyed value" :av "key" k :av "value"
    (mo "cell" :av "value" v
      :av "type" (mo "pointer type" :av "type" et))))
```

3.3.5. Литерал функции

Функциональные литералы создают значения функций, сохраняя текущее состояние замыкания — ячейки памяти всех переменных, видимых в момент создания.

```
(aspect "opsem::rvalue" :context ac :type "function lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do
```

```
(dolist (item ns) (setf c (aset c (aget a "variable cell" item))))
(mo "function value" :av "type"      (aget i "type")
      :av "signature" (aget i "signature")
      :av "body"      (aget i "body")
      :av "closure"   c))
```

3.3.6. Литерал метода

Аналогично создается значение метода.

```
(aspect "opsem::rvalue" :context ac :type "method lit" :instance i
      :agent a :p (aget i "body" "all variables") ns :p nil c :do

(dolist (item ns) (setf c (aset c (aget a "variable cell" item))))
(mo "method value" :av "type"      (aget i "type")
      :av "signature" (aget i "signature")
      :av "body"      (aget i "body")
      :av "closure"   c))
```

3.4. Выражения

3.4.1. Ссылка на переменную

Для ссылки на переменную определены два варианта вычисления: для получения левого значения (lvalue) — ячейки памяти, и для получения правого значения (rvalue) — содержимого ячейки.

```
(aspect "opsem::lvalue" :context ac :type "variable ref" :instance i
      :agent a :do (aget a "variable cell" i))

(aspect "opsem::rvalue" :context ac :type "variable ref" :instance i
      :agent a :do (aget (aget a "variable cell" i) "value"))
```

3.4.2. Выражение в скобках

Выражение в скобках вычисляет свое содержимое.

```
(aspect "opsem::rvalue" :context ac :type "(1)" :instance i :do
  (clear—update—eval—acontext ac :instance (aget i 1)))
```

3.4.3. Приведение типов

Вычисление выражения приведения типов начинается с вычисления значения, которое необходимо привести к другому типу.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage nil :do

  (update—push—acontext ac :stage "type from")
  (clear—update—eval—acontext ac :instance (aget i "value")))
```

На стадии `type from` происходит определение типа вычисленного значения.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "type from" :value tv :ap tv "type" vtp :do

  (update—push—acontext ac :stage "build value")
  (clear—update—eval—acontext ac :av "typed value" tv :stage (nmatch
    :v (is—instance vtp "int type")    T :exit "int type"
    :v (is—instance vtp "float type")   T :exit "float type"
    :v (is—instance vtp "complex type") T :exit "complex type"
    :v (is—instance vtp "string type")  T :exit "string type"
    :v (is—instance vtp "slice type")   T :exit "byte type")))
```

На стадии `int type` рассматривается приведение целого числа к заданному типу. Причем целое число можно привести только к численным типам: знаковые, беззнаковые или вещественные.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "int type" :ap ac "typed value" tv :ap tv "value" v
  :ap i "type" tp :ap tp "bit size" bs
  :p (logand v (1- (ash 1 bs))) x :do

  (nmatch
    :v (is-instance tp "signed int type")    T
    :exit (if (> x (ash 1 (1- bs))) (- x (ash 1 bs)) x)
    :v (is-instance tp "unsigned int type") T :exit x
    :v (is-instance tp "float type")          T :exit (float v 0.0d0)))
```

На стадии `float type` описывается приведение вещественного числа к заданному типу. Причем вещественное число можно привести только к таким же численным типам, что и в целочисленном случае.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "float type" :ap ac "typed value" tv :ap tv "value" v
  :ap i "type" tp :ap tp "bit size" bs
  :p (logand v (1- (ash 1 bs))) x :do

  (nmatch
    :v (is-instance tp "signed int type")    T
    :exit (let ((p (truncate x 1))) (if (> p (ash 1 (1- bs)))
      (- p (ash 1 bs)) p))
    :v (is-instance tp "unsigned int type") T :exit (truncate x 1)
    :v (is-instance tp "float type")          T :exit v))
```

На стадии `complex type` описывается приведение вещественного числа к заданному типу. Причем комплексное число можно привести только к таким же численным типам, что и в двух предыдущих случаях.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
```

```

:stage "complex type" :ap ac "typed value" tv :ap tv "value" v
:p (mo "float type" :av "bit size" (aget i "type" "bit size")) ctp
:p (mo "float type" :av "bit size" (aget tv "type" "bit size")) vtp
:do

(update-push-acontext ac :stage "complex eval im"
  :av "conversion type" ctp :av "im" (mo "typed primitive"
    :av "type" vtp :av "value" (imagpart v)))
(clear-update-eval-acontext ac :instance (mo "conversion"
  :av "type" ctp
  :av "value" (mo "typed primitive" :av "type" vtp :av "value"
    (realpart v))))
)

```

4. Отсюда и далее НЕ правленная версия

На стадии `complex`

```

(aspect "opsem::rvalue" :context ac :type "conversion" :stage "complex
  eval im"
  :ap ac "im" im :ap ac "conversion type" ctp :value re :do
    (update-push-acontext ac :stage "complex return" :av "re" re)
    (clear-update-eval-acontext ac :instance (mo "conversion" :av
      "type" ctp :av "value" im))
)

```

```

(aspect "opsem::rvalue" :context ac :type "conversion" :stage "complex
  return"
  :ap ac "re" re :value im :do (complex (aget re "value") (aget im
    "value"))
)

```

```

(aspect "opsem::rvalue" :context ac :type "conversion" :instance i

```

```

:stage "string type"
:ap ac "typed value" tv :ap tv "value" v :ap i "type" tp :do
(nmatch
:v (is-instance tp "string type") T :exit v
:do (mapcar #'char-code (coerce v 'list)))
)

```

```

(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
:stage "byte type"
:ap ac "typed value" tv :ap tv "value" sv :ap i "type" tp :ap sv
"offset" ofs :do
(let ((lst nil) (cs (aget sv "array" "elements")))) (dotimes (k
ofs) (setf cs (cdr cs)))
(dotimes (k (aget sv "length") (reverse lst)) (setf lst (cons
(agets (car cs) "value") lst)) (setf cs (cdr cs))))
(match :v (is-instance tp "string type") nil :do sv
:exit (coerce (mapcar #'code-char lst) 'string))
)

```

4.0.1. Метод-выражение

Метод-выражение преобразует метод в функцию, где получатель становится первым параметром.

```

(aclosure ac :attribute "opsem::rvalue" :type "method expr" :instance i
:agent a :p (aget a "method value" (aget i "receiver type")
(agets i "name")) mv
:ap mv "type" mt :ap mv "signature" ms :do
(mo "function value"
:av "type" (mo "function type"
:av "param types" (cons (aget mt "receiver type")
(agets mt "param types")))
:av "variadic type" (aget mt "variadic type"))

```

```

      :av "result type"    (aget mt "result types"))
:av "signature" (mo "function signature"
      :av "parameters"    (cons (aget ms "receiver")
                                (aget ms "parameters")))
      :av "variadic param" (aget ms "variadic param")
      :av "result"        (aget ms "result"))
:av "body"        (aget mv "body")
:av "closure" (aget mv "closure"))))

```

4.0.2. Выбор поля или метода

Выбор поля или метода обрабатывает два случая: когда слева от точки стоит имя типа (интерфейса) — возвращается метод; когда слева стоит структура или указатель на структуру — возвращается поле.

```

(aclosure ac :attribute "opsem::rvalue" :type "selector expr"
  :instance i :stage nil :ap i "receiver" r :do
    (update-push-aclosure ac :stage "return value")
    (match :v (and (is-instance r "identifier") (aget a "type" r)) T
      :do (aget a "method value" r (aget i "name"))
      :exit (clear-update-eval-aclosure ac :instance (aget i
        "receiver")))))

```

Для получения левого значения (только для структур).

```

(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "return value")
    (clear-update-eval-aclosure ac :instance (aget i "receiver")
      :attribute "opsem::rvalue"))
(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage "return value"
  :value r :ap i "name" n :ap r "type" tr :do

```

```
(nmatch :v tr "struct type" :exit (aget r "fields" n)
      :v tr "pointer type" :exit (aget r "target" "value"
      "fields" n)))
```

4.0.3. Индексирование

Индексирование применимо к массивам, срезам, картам и строкам.

```
(aclosure ac :attribute "opsem::rvalue" :type "index expr"
  :instance i :stage nil :do
  (update-push-aclosure ac :stage "eval index")
  (clear-update-eval-aclosure ac :instance (aget i "indexable"))))
```

Возврат значения по индексу.

```
(aclosure ac :attribute "opsem::rvalue" :type "index expr"
  :instance i :stage "return value"
  :value index :ap ac "indexable" indl :ap indl "type" it :do
  (nmatch :v it "array type" :exit (aget (nth index (aget indl
    "elements")))
    "value")
    :v it "slice type" :exit (aget (nth (+ (aget indl
    "offset") index)
    (aget indl "array" "elements"))) "value")
    :v it "map type" :exit (aget (aget indl "entries"
    index) "value")
    :v it "string type" :exit (char (aget indl "value") index)))
```

4.0.4. Выражение среза

Выражение среза создает новый срез из существующего массива, среза или строки.

```
(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage nil :do
```

```
(update-push-aclosure ac :stage "eval low")
(clear-update-eval-aclosure ac :instance (aget i "sequence")))
```

Для массива:

```
(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "array type"
  :ap ac "sequence" s :ap ac "low" low :ap ac "high" high :ap ac
  "cap" cap
  :ap s "type" st :p (length (aget s "elements")) len :do
  (if (not high) (aset high len))
  (if (not cap) (aset cap (- len low)))
  (mo "slice value"
    :av "type" (mo "slice type" :av "elem type" (aget st "elem
    type")))
    :av "array" s
    :av "offset" low
    :av "length" (- high low)
    :av "capacity" cap))
```

Для среза (сохраняется ссылка на исходный массив):

```
(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "slice type"
  :ap ac "sequence" s :ap ac "low" low :ap ac "high" high :ap ac
  "cap" cap
  :p (length (aget s "array" "type" "len")) len :do
  (if (not high) (aset high len))
  (if (not cap) (aset cap (- len low)))
  (mo "slice value"
    :av "type" (aget s "type")
    :av "array" (aget s "array")
    :av "offset" (+ (aget s "offset") low))
```

```

:av "length" (aget s "length")
:av "capacity" (aget s "capacity"))))

```

Для строки возвращается подстрока (неизменяемая).

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "string type"
  :p (aget ac "sequence" "value") s :ap ac "low" low :ap ac "high"
  high :do
  (if (not high) (aset high (length s)))
  (subseq s low high))

```

4.1. Унарные выражения

Все унарные выражения сначала вычисляют свой операнд, затем применяют операцию.

```

(aclosure ac :attribute "opsem::rvalue" :type "unary expression"
  :instance i :stage nil :do
  (update-push-aclosure ac :stage "apply")
  (clear-update-eval-aclosure ac :instance (aget i 1)))

```

Вспомогательная функция для приведения целых чисел к нужному типу с учетом переполнения (wrap-around для беззнаковых, знаковое представление для знаковых).

```

(defun convert-int-to-type (tp raw)
  (mo "typed primitive" :av "type" tp :av "value"
    (if (not (is-instance tp "int type")) raw
      (let* ((bs (aget tp "bit size"))
              (v (mod raw (ash 1 bs))))
        (if (or (is-instance tp "unsigned int type")
                (< v (ash 1 (1- bs))))
            v
            (- v (ash 1 (1- bs))))))))

```

Унарные операторы:

```
(aclosure ac :attribute "opsem::rvalue" :type "+1" :stage "apply"
  :value v :do v)
(aclosure ac :attribute "opsem::rvalue" :type "-1" :stage "apply"
  :value pr
  :ap pr "value" v :ap pr "type" tp :do (convert-int-to-type tp (-
    v)))
(aclosure ac :attribute "opsem::rvalue" :type "^1" :stage "apply"
  :value pr
  :ap pr "value" v :ap pr "type" tp :do (convert-int-to-type tp
    (lognot v)))
(aclosure ac :attribute "opsem::rvalue" :type "!1" :stage "apply"
  :value v :do
    (mo "typed primitive" :av "type" (co "bool type")
      :av "value" (not (aget v "value"))))
```

Разыменование указателя (может быть как rvalue, так и lvalue).

```
(aclosure ac :attribute "opsem::rvalue" :type "*1" :stage "apply"
  :value tv :ap tv "value" v :do
    (mo "typed primitive" :av "type" (aget tv "type")
      :av "value" (aget v "value")))
(aclosure ac :attribute "opsem::lvalue" :type "*1" :instance i :stage
  nil :do
    (update-push-aclosure ac :stage "apply")
    (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem::lvalue" :type "*1" :stage "apply"
  :value v :do (aget v "value"))
```

Взятие адреса.

```
(aclosure ac :attribute "opsem::rvalue" :type "&1" :instance i :stage
  "apply")
```

```

:value tv :ap tv "type" tp :ap tv "value" v :do
  (mo "typed primitive" :av "type" (co "pointer type" :av "type" tp)
    :av "value" (mo "cell" :av "value" v :av "type" tp)))

```

4.2. Бинарные выражения

Бинарные выражения вычисляют левый операнд, затем правый (кроме логических операций с коротким замыканием).

```

(aclosure ac :attribute "opsem::rvalue" :type "binary expression"
  :instance i :stage nil :do
  (update-push-aclosure ac :stage "apply 1")
  (clear-update-eval-aclosure ac :instance (aget i 1)))

```

Логические операции с коротким замыканием.

```

(aclosure ac :attribute "opsem::rvalue" :type "1||2" :instance i
  :stage "apply 1" :value v :do
  (match :v (aget v "value") T
    :do (mo "typed primitive" :av "type" (co "bool type") :av
      "value" T)
    :exit (update-push-aclosure ac :stage "apply 2")
      (clear-update-eval-aclosure ac :instance (aget i 2))))
(aclosure ac :attribute "opsem::rvalue" :type "1&&2" :instance i
  :stage "apply 1" :value v :do
  (match :v (aget v "value") nil
    :do (mo "typed primitive" :av "type" (co "bool type") :av
      "value" nil)
    :exit (update-push-aclosure ac :stage "apply 2")
      (clear-update-eval-aclosure ac :instance (aget i 2))))

```

Арифметические и битовые операции. Результат приводится к типу левого операнда.

```
(aclosure ac :attribute "opsem::rvalue" :type "1+2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1+2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (+ fv sv)))
```

```
;; subtraction
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1-2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1-2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (- fv sv)))
```

```
;; multiplication
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1*2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1*2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (* fv sv)))
```

```
;; division
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1/2" :instance i :stage
  "apply 1" :value v :do
```

```

(update-push-aclosure ac :stage "apply 2" :av 1 v)
(clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1/2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (/ fv sv)))

;; remainder
(aclosure ac :attribute "opsem::rvalue" :type "1%2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1%2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (rem fv sv)))

;; left shift
(aclosure ac :attribute "opsem::rvalue" :type "1<<2" :instance i
  :stage "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<<2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (ash fv sv)))

;; right shift
(aclosure ac :attribute "opsem::rvalue" :type "1>>2" :instance i
  :stage "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))

```

```
(aclosure ac :attribute "opsem::rvalue" :type "1>>2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (ash fv (- sv))))
```

```
;; bitwise AND
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1&2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1&2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (logand fv sv)))
```

```
;; bitwise AND NOT
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1&^2" :instance i
  :stage "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1&^2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
  (convert-int-to-type tp (logand fv (lognot sv))))
```

```
;; bitwise OR
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1|2" :instance i :stage
  "apply 1" :value v :do
  (update-push-aclosure ac :stage "apply 2" :av 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1|2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
```

```

sv :do
  (convert-int-to-type tp (logior fv sv)))

;; bitwise XOR
(aclosure ac :attribute "opsem::rvalue" :type "1^2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1^2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (logxor fv sv)))

```

Операции сравнения возвращают типизированное булево значение.

```

(aclosure ac :attribute "opsem::rvalue" :type "1==2" :instance i
  :stage "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1==2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (= fv sv)))

```

```

;; inequality
(aclosure ac :attribute "opsem::rvalue" :type "1!=2" :instance i
  :stage "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1!=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (not (= fv sv))))

```

```
;; less than
(aclosure ac :attribute "opsem::rvalue" :type "1<2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (< fv sv)))
```

```
;; less than or equal
(aclosure ac :attribute "opsem::rvalue" :type "1<=2" :instance i
  :stage "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1<=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (<= fv sv)))
```

```
;; greater than
(aclosure ac :attribute "opsem::rvalue" :type "1>2" :instance i :stage
  "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1>2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (> fv sv)))
```

```
;; greater than or equal
```

```
(aclosure ac :attribute "opsem::rvalue" :type "1>=2" :instance i
  :stage "apply 1" :value v :do
    (update-push-aclosure ac :stage "apply 2" :av 1 v)
    (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem::rvalue" :type "1>=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s :ap s "value"
  sv :do
    (convert-int-to-type tp (>= fv sv)))
```

4.3. Декларации

4.3.1. Декларация типа

Декларация типа добавляет определение типа в окружение агента.

```
(aclosure ac :attribute "opsem" :type "type decl" :instance i :agent a
  :do
    (aset a "type" (aget i "name") (aget i "type")))
```

4.3.2. Декларация переменных

Декларация переменных может происходить двумя способами: с явным указанием значений или без них (используются значения по умолчанию).

```
(aclosure ac :attribute "opsem" :type "var decl" :instance i :stage nil
  :ap ac "block" b :ap i "names" ns :ap i "values" vs :ap i "types"
  ts :do
    (match :v (null vs) T
      :do (update-push-aclosure ac :stage "declarating defaults"
        :av "names" ns :av "types" (cdr ts) :av "block" b)
        (clear-update-eval-aclosure ac :attribute "default value
  by type"
    :instance (car ts)))
```

```

      :exit (update-push-aclosure ac :stage "evaluating values"
        :av "names" ns :av "types" ts :av "values" (cdr vs)
      :av "block" b)
      (clear-update-eval-aclosure ac :instance (car vs))))))

```

4.3.3. Декларация функций и методов

Декларация функции вычисляет значение функционального литерала и сохраняет его в окружении.

```

(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage nil :do
  (update-push-aclosure ac :stage "decl")
  (clear-update-eval-aclosure ac :instance (aget i "value")))
(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage "decl" :value v :agent a :do
  (aset a "function value" (aget i "name") v))

```

4.4. Блоки

Блоки определяют области видимости переменных. При входе в блок сохраняются старые ячейки всех переменных, объявленных в этом блоке.

```

(aclosure ac :attribute "opsem" :type "block" :instance i :stage nil
  :ap i "statements" sts :ap i "decl variables" avs :p nil dv :agent
  a :do
  (aset i "variable cells" (dolist (item avs (reverse dv))
    (setf dv (cons (aget a "variable cell" item) dv))))
  (update-push-aclosure ac :stage "variable back")
  (clear-update-eval-aclosure ac :stage "evaluating statement" :av
    "statements" sts))

```

Вычисление очередного оператора блока.

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "evaluating statement"
  :ap ac "statements" sts :v (null sts) nil :do
    (update-push-aclosure ac :av "statements" (cdr sts))
    (clear-update-eval-aclosure ac :instance (car sts)))
```

Восстановление старых ячеек при выходе из блока.

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "variable back" :do
    (dolist (item (aget i "decl variables"))
      (aset a "variable cell" item (aget item "variable cells")))))
```

4.5. Операторы присваивания

Присваивание сначала вычисляет левую часть (lvalue), затем правую (rvalue).

```
(aclosure ac :attribute "opsem" :type "1=2" :instance i :stage nil :do
  (update-push-aclosure ac :stage "eval 2")
  (clear-update-eval-aclosure ac :instance (aget i 2)))
(aclosure ac :attribute "opsem" :type "1=2" :instance i :stage "eval
  2" :value v :do
  (update-push-aclosure ac :stage "apply" :ac 1 v)
  (clear-update-eval-aclosure ac :instance (aget i 1) :attribute
    "opsem::lvalue"))
(aclosure ac :attribute "opsem" :type "1=2" :stage "apply" :ap ac 1 f
  :value s :do
  (aset f "value" (aget s "value")))
```

Составные присваивания (например, `+=`) выполняют операцию перед записью.

```
(aclosure ac :attribute "opsem" :type "1+=2" :stage "apply" :ap ac 1 f
  :value s :do
```

```
(aset f "value" (convert-int-to-type (aget f "type")
  (+ (aget f "value") (aget s "value")))))
```

4.6. Условные операторы

Условный оператор `if`.

```
(aclosure ac :attribute "opsem" :type "if stmt" :instance i :stage nil
  :do
    (update-push-aclosure ac :stage "apply" :av "block" (aget ac
      "block")))
    (clear-update-eval-aclosure ac :instance (aget i "condition")))
(aclosure ac :attribute "opsem" :type "if stmt" :stage "apply" :value
  c :ap ac "block" b :do
    (clear-update-eval-aclosure ac :instance (aget i (if c "then"
      "else")))
      :av "block" b))
```

4.7. Циклы

Цикл `for condition`.

```
(aclosure ac :attribute "opsem" :type "for condition" :instance i
  :stage nil :ap ac "block" b :do
    (update-push-aclosure ac :stage "apply" :av "block" b)
    (clear-update-eval-aclosure ac :instance (aget i "condition")))
(aclosure ac :attribute "opsem" :type "for condition" :stage "apply"
  :value v :v v T :av ac "block" b :do
    (update-push-aclosure ac :stage nil :av "block" b)
    (clear-update-eval-aclosure ac :instance (aget i "body")))
```

4.8. Операторы передачи управления

Оператор `return`.

```
(aclosure ac :attribute "opsem" :type "return stmt" :instance i :do
  (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem" :type "return stmt" :stage "ret"
  :value v :do (aset a "value" (car v)))
```

4.9. Канальные операции

Оператор отправки в канал.

```
(aclosure ac :attribute "opsem" :type "1<-2" :instance i :do
  (update-push-aclosure ac :stage "send-value")
  (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem" :type "1<-2" :instance i :stage
  "send-value"
  :value ch :ap i 2 e2 :do
  (update-push-aclosure ac :stage "send" :av "channel" ch)
  (clear-update-eval-aclosure ac :instance e2))
(aclosure ac :attribute "opsem" :type "1<-2" :stage "send"
  :ap ac "channel" ch :value val :do
  (aset ch "buffer" (append (aget ch "buffer") (list (aget val
    "value"))))))
```

Операция чтения из канала как выражение.

```
(aclosure ac :attribute "opsem::rvalue" :type "<-1" :instance i :do
  (clear-update-eval-aclosure ac :instance (aget i 1)))
(aclosure ac :attribute "opsem::rvalue" :type "<-1" :stage "receive"
  :value ch :do (pop (aget ch "buffer")))
```